

RAPPORT DE PROJET

Plateforme de supervision IoT en temps réel

Wokwi, ESP32, DHT22, MQTT, Elasticsearch et Kibana

Élément	Information
Réalisé par	Mohamed EL HAMZI
Établissement	À compléter
Filière / Module	À compléter
Encadrant(e)	À compléter
Année universitaire	2025-2026
Date de remise	À compléter

Projet académique de collecte, stockage, visualisation et détection d'anomalies sur des données IoT simulées.

Résumé

Ce projet présente la conception d'une plateforme légère de supervision IoT permettant de simuler un capteur de température et d'humidité, de transmettre ses mesures en temps réel, de les stocker puis de les visualiser. Le capteur DHT22 et la carte ESP32 sont simulés dans Wokwi. Les mesures sont publiées au format JSON sur un broker MQTT public. Un consommateur Python écoute le topic MQTT, valide les messages, ajoute un horodatage et un indicateur d'anomalie, puis indexe les documents dans Elasticsearch. Kibana exploite ensuite cet index afin de construire un tableau de bord composé d'indicateurs et de courbes temporelles.

La détection d'anomalies repose sur des seuils simples et transparents : une température supérieure à 35 °C ou une humidité supérieure à 80 %. Cette approche a été choisie pour obtenir un prototype fonctionnel, compréhensible et démontrable rapidement, tout en conservant une architecture extensible.

Mots-clés : IoT, ESP32, DHT22, Wokwi, MQTT, Python, Elasticsearch, Kibana, supervision, anomalie.

Abstract

This project describes a lightweight real-time IoT monitoring platform. An ESP32 board and a DHT22 temperature and humidity sensor are simulated with Wokwi. Measurements are published as JSON messages to an MQTT broker. A Python consumer subscribes to the MQTT topic, validates incoming data, adds a timestamp and an anomaly flag, and indexes each measurement in Elasticsearch. Kibana is then used to create a monitoring dashboard with current values, historical charts and anomaly indicators. Threshold-based detection identifies temperatures above 35 °C and humidity levels above 80%.

Table des matières

1. Introduction
 2. Analyse du besoin
 3. Conception de la solution
 4. Réalisation
 5. Installation et exécution
 6. Tests et validation
 7. Tableau de bord Kibana
 8. Difficultés rencontrées et solutions
 9. Limites et améliorations
 10. Conclusion
- Annexes

1. Introduction

1.1 Contexte

L'Internet des objets permet à des équipements physiques de collecter et d'échanger des informations. Dans un contexte de bâtiment connecté, la température et l'humidité sont des paramètres importants pour le confort, la sécurité et la surveillance des installations. Une plateforme de supervision doit assurer la collecte continue, le transport, le stockage, la consultation et l'analyse de ces mesures.

1.2 Problématique

Comment construire rapidement une chaîne IoT complète, sans matériel physique, capable de transmettre des mesures en temps réel, de conserver un historique, d'afficher des indicateurs lisibles et de signaler automatiquement les valeurs anormales ?

1.3 Objectifs

- Simuler un ESP32 et un capteur DHT22 dans Wokwi.
- Lire la température et l'humidité toutes les cinq secondes.
- Publier les mesures au format JSON via MQTT.
- Consommer et valider les messages avec Python.
- Indexer les données dans Elasticsearch.
- Construire un tableau de bord Kibana.
- Détecter les dépassements de seuil.

1.4 Périmètre

Le prototype utilise un seul capteur simulé. La détection d'anomalies est basée sur des règles fixes et ne repose pas sur un modèle de Machine Learning. L'authentification et le chiffrement du broker public ne sont pas inclus dans la version de démonstration.

2. Analyse du besoin

2.1 Besoins fonctionnels

ID	Besoin	Critère de réussite
BF-01	Acquérir température et humidité	Valeurs visibles dans le moniteur série
BF-02	Publier une mesure toutes les 5 secondes	Messages reçus régulièrement sur le topic
BF-03	Stocker chaque mesure	Document présent dans l'index iot-data
BF-04	Détecter une anomalie	Champ anomaly renseigné correctement
BF-05	Visualiser les mesures	Courbes et indicateurs visibles dans Kibana

2.2 Besoins non fonctionnels

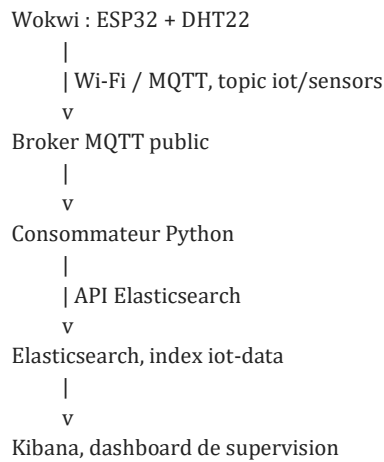
- Simplicité d'installation et d'utilisation.
- Code lisible, commenté et structuré.
- Tolérance aux messages invalides.
- Traçabilité grâce à l'horodatage.
- Architecture modulaire pouvant accueillir d'autres capteurs.

2.3 Cas d'utilisation principal

1. L'utilisateur démarre Elasticsearch et Kibana avec Docker Compose.
2. L'utilisateur lance le consommateur Python.
3. L'utilisateur démarre la simulation Wokwi.
4. L'ESP32 lit le DHT22 et publie les mesures.
5. Le consommateur reçoit, enrichit et indexe les données.
6. L'utilisateur consulte le tableau de bord Kibana.
7. L'utilisateur modifie la température dans Wokwi afin de provoquer une anomalie.

3. Conception de la solution

3.1 Architecture générale



3.2 Rôle des composants

Composant	Rôle
Wokwi	Simule la carte ESP32, le câblage et le capteur DHT22.
ESP32	Lit les mesures et publie les messages MQTT.
Broker MQTT	Assure le transport asynchrone des messages.
Consumer Python	Valide, enrichit et transmet les mesures.
Elasticsearch	Stocke et indexe les documents IoT.
Kibana	Affiche les données et les anomalies.
Docker Compose	Orchestre Elasticsearch et Kibana localement.

3.3 Format des données

```
{
  "device": "esp32-01",
  "temperature": 28.5,
  "humidity": 60.0
}
```

Le consommateur complète ce message avec un horodatage UTC et un indicateur d'anomalie.

```
{
  "device": "esp32-01",
  "temperature": 38.0,
  "humidity": 60.0,
  "timestamp": "2026-09-08T12:00:00Z",
  "anomaly": true,
  "anomaly_reason": "HIGH_TEMPERATURE"
}
```

3.4 Règles d'anomalie

Condition	Résultat
Température > 35 °C	Anomalie de surchauffe
Humidité > 80 %	Anomalie d'humidité élevée
Valeurs sous les seuils	Mesure normale
Message incomplet ou non numérique	Message rejeté et erreur journalisée

4. Réalisation

4.1 Structure du projet

```

iot-monitoring/
├── docker-compose.yml
├── esp32/
│   ├── sketch.ino
│   ├── diagram.json
│   └── libraries.txt
├── consumer/
│   ├── mqtt_consumer.py
│   ├── config.py
│   └── requirements.txt
├── elasticsearch/
│   └── init_index.json
└── README.md
  
```

4.2 Simulation Wokwi

Le capteur DHT22 est relié à l'ESP32 de la manière suivante : VCC vers 3V3, GND vers GND et SDA/DATA vers GPIO 4. La broche NC n'est pas utilisée. Le programme initialise le capteur, se connecte au réseau Wokwi-GUEST, puis publie la température et l'humidité toutes les cinq secondes.

DHT22	ESP32
VCC	3V3
SDA / DATA	GPIO 4
NC	Non connecté
GND	GND

4.3 Communication MQTT

Le topic retenu est `iot/sensors`. Pour une démonstration rapide, Wokwi et le consommateur Python utilisent le même broker MQTT public. Un identifiant de topic plus spécifique peut être utilisé pour éviter les collisions, par exemple `iot/mohamed-elh/esp32-01/sensors`.

4.4 Consommateur Python

Le consommateur s'abonne au topic, convertit le contenu JSON, contrôle la présence et le type des champs, calcule l'état d'anomalie, ajoute l'horodatage puis envoie le document à Elasticsearch. Les exceptions sont journalisées afin qu'un message incorrect n'arrête pas le service.

4.5 Elasticsearch et Kibana

Elasticsearch utilise l'index `iot-data`. Les températures et humidités sont stockées sous forme numérique, le timestamp au format date et le statut d'anomalie au format booléen. Kibana utilise un Data View `iot-data*` basé sur le champ `timestamp`.

5. Installation et exécution

5.1 Prérequis

- Windows 10/11, Linux ou macOS.
- Docker Desktop et Docker Compose.
- Python 3.10 ou version supérieure.
- Accès Internet pour Wokwi et le broker MQTT.
- Navigateur Web récent.

5.2 Démarrage de l'infrastructure

```
cd iot-monitoring
docker compose up -d
docker compose ps
```

Elasticsearch devient accessible sur <http://localhost:9200> et Kibana sur <http://localhost:5601>. Le premier démarrage peut prendre une à deux minutes selon la machine.

5.3 Démarrage du consommateur

```
cd consumer
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python mqtt_consumer.py
```

5.4 Démarrage de Wokwi

8. Créer ou ouvrir un projet ESP32 dans Wokwi.
9. Copier sketch.ino, diagram.json et libraries.txt.
10. Démarrer la simulation.
11. Vérifier dans le moniteur série que les valeurs sont lues et publiées.

5.5 Vérifications

```
curl http://localhost:9200
curl "http://localhost:9200/iot-data/_search?pretty"
```

Dans Kibana : Stack Management > Data Views > Create data view, puis saisir iot-data* et sélectionner timestamp comme champ temporel.

6. Tests et validation

6.1 Stratégie de test

Les tests valident séparément l'acquisition, le transport, le traitement, le stockage et l'affichage. La démonstration principale consiste à modifier les valeurs du DHT22 dans Wokwi et à suivre leur propagation jusqu'à Kibana.

Test	Entrée / action	Résultat attendu
T1 Lecture normale	25 °C, 60 %	Valeurs affichées dans le moniteur série
T2 Publication MQTT	Démarrer Wokwi	Message reçu par le consumer
T3 Indexation	Message valide	Document créé dans iot-data
T4 Température élevée	40 °C, 60 %	anomaly = true, HIGH_TEMPERATURE
T5 Humidité élevée	25 °C, 90 %	anomaly = true, HIGH_HUMIDITY
T6 Message invalide	Champ temperature absent	Message rejeté sans arrêt du consumer
T7 Visualisation	Ouvrir Discover	Document visible avec timestamp

6.2 Scénario de démonstration

12. Régler le DHT22 à 25 °C et 60 %. Vérifier que la mesure est normale.
13. Régler le DHT22 à 40 °C. Attendre cinq secondes.
14. Observer le message d'anomalie dans le terminal du consommateur.
15. Actualiser Kibana et vérifier l'apparition du point à 40 °C et du statut anomaly=true.
16. Revenir à une valeur normale et vérifier la continuité de la courbe.

7. Tableau de bord Kibana

7.1 Visualisations proposées

Visualisation	Configuration suggérée
Température actuelle	Metric, dernière valeur de temperature
Humidité actuelle	Metric, dernière valeur de humidity
Historique température	Line chart, timestamp en X et moyenne de temperature en Y
Historique humidité	Line chart, timestamp en X et moyenne de humidity en Y
Nombre d'anomalies	Metric, filtre anomaly:true
Répartition normal/anormal	Donut, segmentation sur anomaly

7.2 Organisation du dashboard

La première ligne contient les indicateurs actuels. La deuxième ligne présente les courbes temporelles. La troisième ligne regroupe le nombre d'anomalies et leur répartition. Un filtre sur device permet de préparer l'ajout de plusieurs capteurs.

7.3 Captures d'écran à insérer

- Figure 1 : câblage ESP32 et DHT22 dans Wokwi.
- Figure 2 : moniteur série avec les valeurs publiées.
- Figure 3 : terminal du consommateur Python.
- Figure 4 : documents de l'index dans Kibana Discover.
- Figure 5 : tableau de bord final.
- Figure 6 : anomalie de température à 40 °C.

Note : remplacer cette liste par les captures réelles du projet avant la remise du rapport.

8. Difficultés rencontrées et solutions

Difficulté	Cause	Solution appliquée
DHTesp.h introuvable	Bibliothèque absente	Ajouter DHTesp dans libraries.txt.
Valeurs nan	GPIO incorrect ou lecture invalide	Relier SDA à GPIO 4 et utiliser le même numéro dans le code.
Docker non reconnu	Docker Desktop absent ou PATH non chargé	Installer Docker Desktop, redémarrer, vérifier docker --version.
Wokwi ne joint pas localhost	Wokwi s'exécute dans le cloud	Utiliser un broker MQTT public commun à Wokwi et Python.
git add échoue sur iotprojet	Dépôt Git imbriqué sans commit	Supprimer le .git interne ou configurer un vrai sous-module.
Kibana ne montre rien	Data View ou champ temporel incorrect	Créer iot-data* et choisir timestamp.

8.1 Points de vigilance

- Utiliser un topic MQTT unique pour éviter de recevoir les messages d'autres utilisateurs.
- Démarrer le consommateur avant Wokwi pour observer tous les messages.
- Attendre qu'Elasticsearch soit sain avant de lancer l'indexation.
- Ne pas publier d'identifiants sensibles dans un dépôt Git.

9. Limites et améliorations

9.1 Limites actuelles

- Le broker public n'offre pas les garanties d'un environnement de production.
- Les seuils sont fixes et ne tiennent pas compte du contexte.
- Un seul type de capteur est simulé.
- La reprise après une longue indisponibilité d'Elasticsearch est limitée.
- Le dashboard Kibana est configuré manuellement.

9.2 Améliorations possibles

- Utiliser Mosquitto privé avec TLS et authentification.
- Ajouter les capteurs de fumée et de mouvement PIR.
- Détecter la perte de connexion lorsqu'aucune mesure n'est reçue pendant 30 secondes.
- Ajouter une file de reprise locale dans le consommateur.
- Créer automatiquement le Data View et le dashboard Kibana.
- Utiliser Centreon pour surveiller la disponibilité du consumer, d'Elasticsearch et du capteur.
- Remplacer les seuils fixes par une détection statistique ou un modèle de Machine Learning.

10. Conclusion

Le projet a permis de mettre en place une chaîne IoT complète allant de la simulation d'un capteur jusqu'à la visualisation des données. Wokwi facilite la validation du câblage et du programme ESP32 sans matériel physique. MQTT assure un transport léger et découplé. Le consommateur Python constitue la couche de traitement, tandis qu'Elasticsearch fournit le stockage indexé et Kibana la supervision graphique.

La solution répond aux objectifs principaux : collecte périodique de température et d'humidité, transmission en temps réel, historisation, visualisation et détection d'anomalies. Malgré son périmètre volontairement limité, l'architecture est modulaire et peut évoluer vers plusieurs capteurs, une surveillance Centreon, des alertes externes et une détection plus intelligente.

Ce prototype montre qu'une plateforme IoT démontrable peut être réalisée rapidement en combinant des outils accessibles, tout en conservant les principes essentiels d'une architecture de supervision moderne.

Annexe A - Commandes utiles

```
# Vérifier Docker
docker --version
docker compose version

# Démarrer les services
docker compose up -d
docker compose ps

# Consulter les journaux
docker compose logs -f elasticsearch
docker compose logs -f kibana

# Arrêter les services
docker compose down

# Vérifier Elasticsearch
curl http://localhost:9200
curl "http://localhost:9200/_search?pretty"
```

Annexe B - Checklist de remise

- ☐ Le code Wokwi compile sans erreur.
- ☐ Le DHT22 retourne des valeurs numériques.
- ☐ Le consumer reçoit les messages MQTT.
- ☐ Les documents sont présents dans Elasticsearch.
- ☐ Le Data View Kibana est correctement configuré.
- ☐ Le dashboard contient les six visualisations recommandées.
- ☐ Une anomalie à 40 °C a été démontrée.
- ☐ Les captures d'écran sont ajoutées au rapport.
- ☐ Les informations de couverture sont complétées.
- ☐ Le dépôt Git ne contient ni .env ni secrets.